

AutoVerifier: Residual-Guided Non-Parametric Optimization for Reference-Based Answer Verification

Zebei Zhao¹, Zhihao Shi¹, Minqi Shi²

¹University of Science and Technology of China

²Beihang University

{zbzhao, zhihaoshi}@mail.ustc.edu.cn

Sherrie@buaa.edu.cn

Abstract

Reference-based verifiers are important for evaluating reasoning models and providing accurate outcome rewards in reinforcement learning with verifiable rewards. To improve verification accuracy, prior work has explored rule-based, model-based, and tool-augmented verifiers for checking answer equivalence across diverse answer forms. However, the equivalence of answer forms such as $1 + 3.14$ and $1 + \pi$ may depend on the question and scoring criterion. We frame such implicit assumptions as verifier inductive biases. To address this challenge, we propose AUTOVERIFIER, a residual-guided non-parametric optimization method that learns these biases from recurring verifier errors. Specifically, AUTOVERIFIER records these biases in rule cards and promotes them to code modules or prompt guidance only after replay validation detects no direct regressions, keeping accepted updates auditable, editable, and reusable. Experiments on four verifier benchmarks demonstrate that AUTOVERIFIER outperforms state-of-the-art verifiers by a large margin.

1 Introduction

Answer verification is widely used to check outputs of reasoning models in benchmark evaluation (Yan et al., 2025; Chen et al., 2025a; Liu et al., 2025a) and to provide outcome rewards for reinforcement learning with verifiable rewards (Shao et al., 2024; DeepSeek-AI, 2025; Yu et al., 2025). In reference-based verification, a verifier takes a question, a reference answer, and a candidate response as input to determine the correctness of the candidate response. As reasoning models produce longer and more diverse responses, exact matching alone is insufficient; the verifier must judge answer equivalence across diverse answer forms.

Recent verifier work has improved either answer matching coverage or made parts of comparison

Question: 6 black / 2 red marbles; draw 3; P(2 black)?	Question: Ellipse area, major axis 9 and minor axis 4.	Question: Force near Ceres, answer requested in N.
Reference Answer: 0.5357	Reference Answer: 28.27	Reference Answer: $3.71e-2\text{ N}$
Candidate Answer: 0.54	Candidate Answer: 9π	Candidate Answer: 0.037 N



Rule-based



Model-based



Tool-augmented



AutoVerifier

Figure 1: Answer equivalence can depend on implicit assumptions. The examples cover rounded probabilities, symbolic forms for rounded numeric references, and scientific notation with units. Whether each pair is equivalent depends on the question and scoring criterion.

logic more explicit. Rule-based verifiers and libraries such as Math-Verify make comparison logic inspectable as code for fixed checks such as symbolic equality and numerical tolerances (Kydliček, 2025). Model-based verifiers and judge models improve coverage for challenging answer matching cases such as formulas, sequences, and multi-part answers, with recent reward models using generated rationales or critiques to support judgment (Chen et al., 2025a; Liu et al., 2025a; Chen et al., 2025b; Liu et al., 2025b; Xia et al., 2025). Tool-augmented verifiers add executable computation for scientific questions (Feng et al., 2025).

Despite these advances, a key gap remains when human annotations reflect implicit assumptions about which candidate answers should count as equivalent for a given question and scoring criterion. Figure 1 illustrates this issue with rounded probabilities, symbolic forms for rounded numeric references, and scientific notation with units. For example, $1 + 3.14$ is not equivalent to $1 + \pi$ in an exact computation problem, but can be acceptable in an applied numerical problem where a rounded value of π is expected. Following Baxter (2000), we frame these implicit assumptions as verifier in-

ductive biases to be learned from recurring verifier errors. Rule-based and tool-augmented verifiers remain limited by fixed comparison logic. Recent model-based verifier work covers complex answer matching, but treats ambiguous numerical acceptability thresholds as outside the binary verification setting (Liu et al., 2025a). Moreover, criteria learned by model-based verifiers remain embedded in model behavior rather than exposed as editable comparison logic. The unresolved problem is therefore to learn verifier inductive biases automatically from evidence while keeping accepted updates auditable, editable, and reusable.

To address this problem, we propose AUTOVERIFIER, a non-parametric optimization method that learns verifier inductive biases from recurring verifier errors. Specifically, AUTOVERIFIER groups recurring errors, records the corresponding biases in rule cards with supporting examples and counterexamples, and promotes a card only after evidence and replay validation confirm no direct regressions. Cards with deterministic comparison logic become code modules after replay validation, while cards that require model judgment are incorporated into prompt guidance for the model-based fallback.

Experiments on four benchmarks demonstrate that AUTOVERIFIER outperforms existing reference-based verifier baselines. AUTOVERIFIER reaches 93.05% macro accuracy, exceeding the reported accuracy of rule-based, model-based, scientific, and tool-augmented verifiers. Compared with the prompt-only setting, adding code modules improves macro accuracy by +1.76 points, indicating that learned comparison logic contributes beyond prompt rewriting. The same direct decisions also reduce fallback calls by 32.13%.

Our key contributions can be summarized as follows:

- We identify implicit annotation assumptions as **verifier inductive biases** that can be learned from recurring verifier errors.
- We propose **AUTOVERIFIER**, a residual-guided non-parametric optimization method that records recurring verifier errors as rule cards and promotes them through replay validation.
- We conduct experiments on four reference-based verification benchmarks, demonstrating that AUTOVERIFIER improves over a prompt-only baseline, outperforms reported verifier baselines, and reduces fallback model calls.

2 Related Work

Answer verifiers. Answer verifiers commonly rely on predefined rules, model judgments, or tool pipelines. Rule libraries such as Math-Verify encode extractors, parsers, normalizers, numeric tolerances, symbolic equivalence checks, and abstention conditions as inspectable procedures (Kydlíček, 2025); scientific verifiers add domain logic for algebraic equivalence, unit conversion, and notation handling (Zheng et al., 2025b). This auditability still requires manual coverage growth, and subtle equivalence remains failure prone (Huang et al., 2025). Model-based verifiers broaden coverage beyond fixed answer matching rules (Chen et al., 2025a; Liu et al., 2025a; Chen et al., 2025b; Liu et al., 2025b; Xia et al., 2025), while tool-augmented verifiers such as CoSineVerifier execute computation or unit conversion for scientific questions (Feng et al., 2025). However, rule and tool pipelines still rely on fixed equivalence checks, while model-based acceptance criteria remain embedded in model behavior. AUTOVERIFIER targets the middle ground by recording recurring verifier errors in rule cards that become auditable code modules, while unresolved cases remain with the model-based fallback.

Non-parametric updates from experience. The learning beyond gradients perspective studies systems that improve by editing external state, including code, tests, memory, and evaluation records, rather than model parameters (Weng, 2026). Procedural memory work such as Skill-Pro promotes reusable procedures through activation, execution, termination, and validation (Mi et al., 2026); related agents and program search systems evolve skills, tools, scaffolds, or code from experience (Wang et al., 2023; Hao et al., 2026; Zheng et al., 2025a; Romera-Paredes et al., 2024; Novikov et al., 2025). AUTOVERIFIER instantiates this non-parametric update pattern for verification, where recurring verifier errors are recorded as rule cards and accepted cards become code modules or prompt guidance after replay validation.

3 Method

AUTOVERIFIER learns verifier inductive biases from recurring verifier errors with a model-based fallback verifier. Construction replay exposes answer forms that the current verifier mishandles, and accumulated errors are grouped into recurring pat-

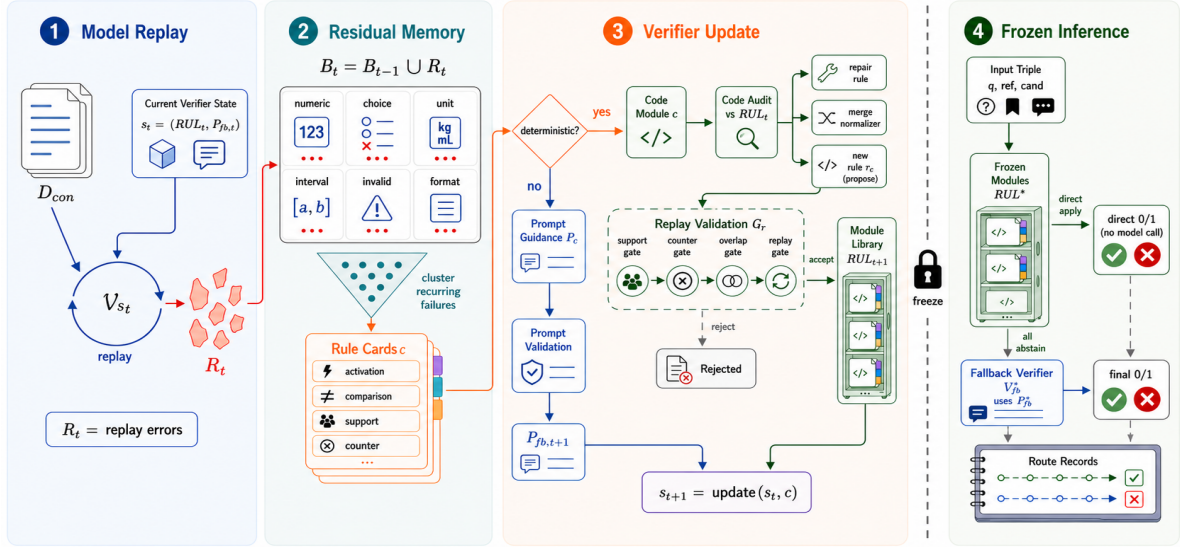


Figure 2: Overview of AUTOVERIFIER. During construction, the current verifier is replayed on 5,000 examples, and recurring verifier errors are recorded as rule cards. Cards with deterministic comparison logic enter code audit and replay validation before becoming code modules, while cards that require model judgment are incorporated into prompt guidance. After freezing, code modules make direct decisions when they apply, and examples for which all code modules abstain go to the model-based fallback.

terns before any update is proposed. A recurring pattern becomes a rule card that records the bias, its support examples, its counterexamples, and the conditions under which a code module should abstain. Here, a code module is a deterministic verifier unit that can return CORRECT, INCORRECT, or ABSTAIN. Cards with deterministic comparison logic enter code audit and replay validation before becoming code modules, while cards that require model judgment are incorporated into prompt guidance. This section defines the verification setting, the verifier update, the rule card paths, the replay validation procedure, and the frozen inference procedure.

3.1 Problem Formulation

We focus on pointwise reference-based answer verification. Each example is

$$x_i = (q_i, a_i^*, \hat{a}_i, m_i),$$

where q_i is a question, a_i^* is the reference answer, $\hat{a}_i$ is a candidate response, and m_i contains optional metadata such as domain or answer type. The binary label $y_i \in \{0, 1\}$ indicates correctness (CORRECT or INCORRECT).

The construction problem starts from a fixed base verifier V_{base} and updates the verifier around it. We denote the construction pool by D_{con} . A residual is a construction example where the current verifier disagrees with y_i , and these residuals

provide evidence for rule extraction. Support examples and counterexamples are construction examples used to validate a proposed rule card before benchmark evaluation. A recurring error pattern is treated as evidence that the current verifier lacks a verifier inductive bias for deciding whether a recurring answer transformation is equivalent or nonequivalent. Patterns that can be expressed as deterministic code become updates to a **module library** of code modules after code audit and replay validation, while patterns that require model judgment are handled by a **model-based fallback verifier** V_{fb} implemented by the fixed model with fallback prompt P_{fb} .

3.2 Verifier State Updates

At construction round t , the verifier state is $s_t = (\text{RUL}_t, P_{\text{fb},t})$, where RUL_t is the module library and $P_{\text{fb},t}$ is the fallback prompt. Replaying this state on construction data yields the residual set:

$$R_t = \{x_i \in D_{\text{con}} : V_{s_t}(x_i) \neq y_i\}.$$

The residual buffer accumulates recurring errors before rule card drafting. Let V_{s_t} denote the verifier induced by s_t under the fixed routing rule. We maintain a construction residual buffer $\mathcal{B}_t = \bigcup_{\tau \leq t} R_\tau$. Here R_t is the residual set for the round, while $\mathcal{B}_t$ stores accumulated residuals used for later clustering. The coordinator groups recurring error patterns in $\mathcal{B}_t$ by answer object, surface form,

source domain, and observed error type before drafting rule cards.

Rule cards specify how a bias should be applied. Each card records the answer comparison procedure, activation region, abstention behavior, and support examples and counterexamples that validate the proposal. A card with deterministic comparison logic proposes a code update r_c , which may repair or merge an existing module path or add a new code module when the behavior is not already covered; a card that requires model judgment proposes prompt guidance $P_c = \text{Guide}(P_{\text{fb},t}, c)$.

The promotion check serves as conservative validation because a candidate may expand direct decisions only if replay shows no direct regression on protected construction examples. Let $G_r(c; s_t)$ be the conjunction of support, counterexample, overlap, and full replay validation for a code module candidate, and let $G_p(c; s_t)$ be the validation check for prompt guidance. Let $\mathcal{C}_{\text{code}}$ and $\mathcal{C}_{\text{guide}}$ denote code and guidance cards. We write $s_t \oplus r_c$ for applying a code update that passes replay validation to RUL_t , and $s_t \oplus P_c$ for revising $P_{\text{fb},t}$ with guidance from card c . The state update is

$$s_{t+1} = \begin{cases} s_t \oplus r_c, & c \in \mathcal{C}_{\text{code}}, G_r(c; s_t), \\ s_t \oplus P_c, & c \in \mathcal{C}_{\text{guide}}, G_p(c; s_t), \\ s_t, & \text{otherwise.} \end{cases}$$

Thus, the construction loop uses a proposal followed by conservative validation, while updating verifier state around a fixed model rather than model parameters.

Replay validation objective. The replay validation objective formalizes conservative promotion as supported coverage under zero direct error constraints. Let $V_0 = V_{\text{base}}$. In the first construction round, residuals are collected from the fixed base verifier:

$$R_0 = \{x_i : x_i \in D_{\text{con}}, V_{\text{base}}(x_i) \neq y_i\}.$$

In later rounds, residuals are collected by replaying the current verifier state $(\text{RUL}_t, P_{\text{fb},t})$ on construction data. Let $K_{\text{RUL}_t}(x) \in \{0, 1, \perp\}$ be the direct output of the current module library, where $\perp$ means that all activated code modules abstain. For a candidate rule card, let S^+ denote positive support examples, S^- denote counterexamples, and C denote the larger construction replay pool. AUTOVERIFIER constructs code modules RUL to

maximize supported direct coverage under replay constraints with no regressions:

$$\text{Cov}_{S^+}(\text{RUL}) = \frac{1}{|S^+|} \sum_{x_i \in S^+} \mathbf{1}[K_{\text{RUL}}(x_i) \neq \perp].$$

$$\text{Err}_A(\text{RUL}) = \sum_{x_i \in A} \mathbf{1}[K_{\text{RUL}}(x_i) \neq \perp \wedge K_{\text{RUL}}(x_i) \neq y_i].$$

$$\begin{aligned} \max_{\text{RUL}} \quad & \text{Cov}_{S^+}(\text{RUL}) \\ \text{s.t.} \quad & \text{Err}_{S^+ \cup S^-}(\text{RUL}) = 0, \\ & \text{Err}_C(\text{RUL}) = 0. \end{aligned}$$

Support examples encourage intended coverage, while counterexamples and replay examples require the code module to abstain or return the construction label on nearby negatives. In this sense, the promotion check serves as conservative validation because a candidate may add supported direct decisions but cannot introduce direct regressions on protected construction replay. The fallback verifier handles abstained or novel cases after the verifier is fixed.

3.3 Rule Cards and Verifier Updates

AUTOVERIFIER treats residuals as evidence of recurring verifier error patterns. Repeated residuals are summarized into structured profiles containing source, domain, answer object, candidate surface form, observed label mismatch, and short error hints. The coordinator first groups profiles by answer object and observed error type, then filters out groups defined only by benchmark identity or a single example. Only after this clustering step are surviving groups written as rule cards. Each rule card is a structured proposal with activation, comparison procedure, abstention rule, support examples, counterexamples, and replay evidence.

- Rule cards with deterministic activation and comparison logic are compiled into **code modules**, which repair an existing module path or add a new direct decision path to the reusable library.
- Rule cards that cannot be encoded safely as deterministic code are incorporated into prompt guidance, which updates the textual prompt processed by the fixed model but never makes direct decisions.

This proposal and replay loop implements the objective above. Accepted code modules expand supported direct coverage while preserving the replay constraints.

Code module and prompt guidance paths. AUTOVERIFIER implements two update paths. A rule card with deterministic logic first enters code audit against the current module library. A missing guard or overbroad normalizer triggers repair or merge of the existing module path, while uncovered behavior triggers a new code module. Cards that require model judgment are incorporated into prompt guidance, preserving lessons useful for judgment but unsafe for direct deterministic decisions. A deterministic coordinator applies the checks above and stores rejected candidates for later rounds.

3.4 Replay Validation for Promotion

This subsection defines the replay validation used to promote a candidate code module. Each candidate code module must expose activation, execution, and termination structure, specialized to binary answer verification:

- **Activation condition** α_r : defines when the direct comparison may run.
- **Execution procedure** π_r : performs the deterministic answer comparison procedure.
- **Termination rule** τ_r : returns CORRECT, INCORRECT, or ABSTAIN.

Candidate code modules undergo four construction data checks:

- **Support:** target support examples must trigger the intended activation path and return the construction label.
- **Counterexamples:** nearby negative examples must abstain or return the construction label.
- **Overlap:** the candidate must add coverage rather than duplicate existing checks.
- **Replay:** full construction replay must not introduce new direct errors.

The checks enforce narrow activation, nearby counterexamples, and replay protected promotion. Accepted updates are compiled into deterministic code, while failed candidates are stored to avoid repeated proposals. Support examples come from recurring verifier errors or fixtures that exercise the intended activation path. Counterexamples are nearby construction cases that share surface form but differ in answer object, polarity, unit, interval endpoint, terminal slot, or invalid output status.

Numeric tolerances, unit normalizers, terminal extraction windows, and closed vocabulary sets are written into the rule card during construction and can be changed only through construction replay. If a candidate changes a construction example that the prompt labels correctly into a direct error, duplicates an earlier check without verified gain, or depends on an unchecked semantic assumption, it is rejected.

Appendix A gives a concrete rule card example, including activation, execution, abstention, counterexamples, and promotion evidence.

3.5 Frozen Inference and Construction Trace

AUTOVERIFIER freezes the code modules, prompt, parser, and routing order before benchmark evaluation. This subsection defines how direct decisions and the model-based fallback are combined after construction.

Fallback verifier. The model-based fallback verifier V_{fb} uses the final AUTOVERIFIER sectioned binary prompt P_{fb} produced during construction. It handles examples for which no validated code module can make a direct decision. Prompt optimization uses recurring verifier errors, accepted rule cards, and rejected rule feedback to update prompt guidance while preserving the same binary interface as the official benchmark prompts. Each call receives a question, a reference answer, and a candidate answer, then emits CORRECT or INCORRECT. This design keeps recurring verifier errors that are not safe to encode as deterministic code available as prompt guidance while reserving deterministic promotion for rule cards with support examples.

Inference and routing. At inference, activated code modules run first in a fixed priority order set before evaluation. Let r^* be the first activated code module that returns a direct decision. If such a module outputs CORRECT or INCORRECT, V_{fb} is not invoked, and later modules cannot override the decision. During construction replay, any candidate whose direct decisions conflict with an earlier promoted module on overlapping support or replay examples is rejected unless it abstains on the overlap or the coordinator updates the fixed priority order and replays the full construction pool. Only when all code modules abstain is the **fallback verifier** V_{fb} consulted:

Verifier	Accuracy (%)				Avg.
	VerifyBench	VerifyBench-Hard	SCI-VerifyBench	VerifierBench	
<i>Rule-based verifiers</i>					
Exact Match	50.55	70.76	46.68	61.52	57.38
Math-Verify	66.95	76.00	60.32	59.92	65.80
<i>Model-based verifiers</i>					
XVerify-8B-I	91.95	83.10	80.16	80.80	84.00
CompassVerifier-32B	92.55	87.90	86.24	90.80	89.37
<i>Scientific and tool-augmented verifiers</i>					
SCI-Verifier-8B	–	90.30	86.28	93.01	89.86*
CoSineVerifier-Label-32B	95.70	90.00	86.40	–	90.70*
CoSineVerifier-Tool-4B	96.60	91.90	89.70	–	92.73*
<i>Same GPT-5.4-Mini verifier interface</i>					
Official prompt	96.30	90.29	86.76	90.77	91.03
AUTOVERIFIER Prompt Only	96.20	90.49	86.80	91.66	91.29
AUTOVERIFIER Prompt + Code	96.75	92.43	89.92	93.11	93.05

Table 1: Results of reference-based answer verification benchmarks. The local variants use the same GPT-5.4-Mini binary verifier interface; Prompt + Code adds frozen code modules before the same AUTOVERIFIER prompt verifier. Avg. averages reported columns; starred averages use fewer than four benchmarks.

$$V_{AV}(x) = \begin{cases} \tau_{r^*}(\pi_{r^*}(x)), & \text{if } r^* \text{ exists,} \\ V_{fb}(x), & \text{otherwise.} \end{cases}$$

This design keeps deterministic decisions guarded and validated by replay while maintaining flexibility for complex or ambiguous cases.

The construction process also records a round trace that is evaluated after the runtime state is frozen.

4 Experiments

We evaluate AUTOVERIFIER under a frozen construction protocol to test whether recurring verifier errors can be converted into reliable verifier inductive biases. Across four reference-based verification benchmarks, we compare with reported rule-based, model-based, scientific, and tool-augmented verifiers, then use prompt controls, construction replay, direct decision audits, and failure analysis to isolate the contribution of accepted code modules.

4.1 Experimental Setup

Construction data. We build AUTOVERIFIER from a construction pool of 5,000 examples designed to expose recurring verifier errors before benchmark evaluation. The pool combines verifier examples from VAR and the TIGER-Lab verifier-sft-data dataset, pairs of questions and reference answers from SuperGPQA and WebInstruct, and constructed diagnostic examples, all normalized into the verifier schema $(q, a^*, \hat{a}, y)$. We select examples to cover answer objects and recurring error

types, including numeric and unit normalization, symbolic or interval comparison, terminal extraction, closed choice alignment, string or sequence verification, and invalid output cases. During construction, this pool provides residuals, support examples, and counterexamples for learning verifier inductive biases, with source counts and decontamination checks reported in Appendix B.2.

Evaluation benchmarks. We evaluate the frozen verifier on four reference-based verification benchmarks: VerifyBench and VerifyBench-Hard (Yan et al., 2025), SCI-VerifyBench (Zheng et al., 2025b), and VerifierBench (Liu et al., 2025a). These benchmarks span general answer verification, hard final answer cases, scientific answer checking, and broad LLM output verification.

Baselines. We group baselines by verification style. Rule-based verifiers include Exact Match (Yan et al., 2025) and Math-Verify (Kydlíček, 2025). Model-based verifiers include XVerify (Chen et al., 2025a) and CompassVerifier (Liu et al., 2025a). Scientific and tool-augmented verifiers include SCI-Verifier (Zheng et al., 2025b) and CoSineVerifier (Feng et al., 2025). For controlled attribution, we also include GPT-5.4-Mini variants under the same binary verifier interface.

Metrics. We use accuracy as the primary metric. We report macro accuracy across benchmarks and use micro accuracy weighted by dataset size for the controlled prompt comparison. We also report code module coverage, fallback verifier accuracy on examples for which all code modules abstain,

fallback call reduction, and direct decision audits from evaluation records.

Construction and freeze protocol. We separate construction from benchmark evaluation by freezing every verifier component before scoring. During construction, we use GPT-5.5 to propose rule cards, code module candidates, and prompt guidance from the construction pool, while GPT-5.4-Mini runs the current fallback verifier during construction replay whenever all active code modules abstain. We admit candidate updates only after structured parsing and replay validation. Before benchmark evaluation, we freeze the module library, fallback prompt, parser, module priority order, and evaluation scripts. During benchmark evaluation, the same GPT-5.4-Mini fallback verifier is called only for examples for which all frozen code modules abstain. Appendix B reports construction data, support statistics, decontamination checks, model configuration, and LLM call accounting.

4.2 Main Results

Benchmark comparison. We demonstrate that AUTOVERIFIER consistently outperforms rule-based, model-based, scientific, and tool-augmented verifiers across four answer-verification benchmarks. Table 1 reports the full accuracy comparison, where AUTOVERIFIER achieves the best result on every benchmark. Relative to the best compared baseline on each benchmark, AUTOVERIFIER improves accuracy by +0.15 on VerifyBench, +0.53 on VerifyBench-Hard, +0.22 on SCI-VerifyBench, and +0.10 on VerifierBench. The final frozen verifier reaches 93.05 macro accuracy across the four benchmarks.

Controlled prompt comparison. We attribute most of the controlled gain to adding code modules, whereas prompt rewriting alone gives only a small mixed improvement. All prompt controls use GPT-5.4-Mini and the same binary verifier interface. We replay the public benchmark prompt format in the official prompt setting, and we use the sectioned binary prompt produced during construction in the AUTOVERIFIER prompt setting. For Prompt + Code, we keep the fallback model, final prompt, parser, and binary interface fixed, adding only code modules before the prompt verifier. Prompt rewriting raises macro accuracy from 91.03 to 91.29, while adding code modules raises it to 93.05 with positive gains on every benchmark. The +1.76 macro point gain also holds under micro

accuracy weighted by dataset size, where the three local variants score 90.84, 91.15, and 92.95.

4.3 Construction Replay Trace

Round coverage. We demonstrate that accepted construction rounds expand supported direct coverage progressively as recurring residuals are converted into validated code modules. We evaluate the verifier state after each accepted round on the construction monitor and, for reference, on the four benchmarks in Figure 3. We interpret the stepwise increases as evidence that the final verifier is built through accumulated rule promotion rather than a single late update.

Accepted rule groups. We analyze the construction trace by accepted rule groups and observe that early rounds account for most covered construction examples while later rounds add smaller, more specialized updates. By Round 10, the accepted construction path covers 2,109 examples in the construction monitor, which contains 5,000 examples. Rounds 1–5 account for 1,840 of these covered examples, and Rounds 6–10 add 269 more. This split across rounds is also reflected in the accepted groups, with extraction and normalization rules entering earlier and sequence, invalid-output, and boundary rules entering later. Appendix Tables 8 and 9 report the full construction audit and one revision case.

4.4 Direct Decision Attribution

Direct coverage and corrections. We observe that code modules both make direct decisions and correct errors from the prompt-only verifier. Table 2 reports 2,665 direct decisions, corresponding to 32.13% fewer fallback calls, and 149 prompt-only verifier errors corrected by code modules. Direct coverage is substantial across all four benchmarks (26.65%–38.04%), while corrections are largest on SCI-VerifyBench (78), where the fallback verifier is weakest on abstained examples.

Coverage versus corrections. We interpret direct coverage and accuracy contribution as related signals that are not interchangeable. Corrections are larger where the fallback prompt is weaker, while direct coverage also measures routing. We therefore treat fallback call reduction as a consequence of direct routing, not as the main evidence for inductive bias acquisition. The appendix direct decision audit reports zero benchmark label

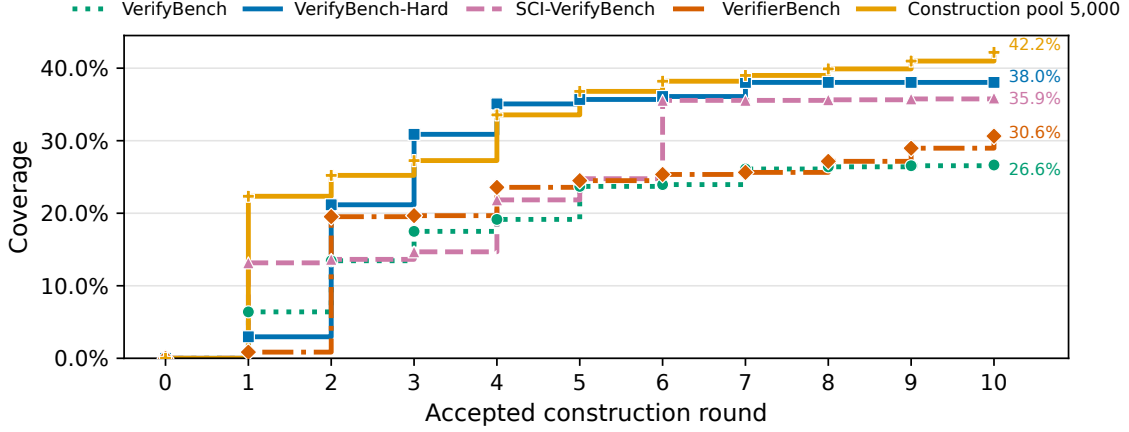


Figure 3: Per round replay trace. Round 0 is the baseline without code modules and Round 10 is the frozen verifier. Curves plot cumulative direct coverage on four evaluation benchmarks and the frozen construction monitor of 5,000 examples.

Bench.	Rules	Direct	Direct%	Corr.	Fallback
VB	29	533	26.65	11	95.57
Hard	31	372	38.04	19	87.79
SCI	41	897	35.88	78	84.28
VerifierBench	24	863	30.64	41	90.07
All	—	2,665	32.13	149	89.61

Table 2: Direct decision attribution. Direct% is the share of examples handled by code modules, equal to fallback call reduction; Corr. counts prompt errors fixed by code modules. Fallback is prompt verifier accuracy on examples for which all code modules abstain.

disagreements for direct decisions made by code modules.

Negative controls. We test generic shortcut rules as an alternative explanation, but the controls do not reproduce the gains from code modules. A small generic heuristic control covers 502 examples but introduces 15 regressions, and promotion using support alone admits replay and counterexample violations. We report these controls and fallback diagnostics in Appendix C.2, and construction call accounting in Appendix D.2.

4.5 Bias Categories and Failure Boundaries

Bias categories. We find that learned verifier inductive biases concentrate around recurring answer forms and guard conditions tied to implicit annotation assumptions, specifying when common answer forms should be treated as equivalent or nonequivalent.

Accepted rule categories. We observe that no single rule group explains the full improvement, but each accepted group contributes a guarded portion of the gain. Table 3 reports direct decisions, corrected prompt-only verifier errors, and macro

accuracy drops for each removed group. These removal results indicate that the gains come from reusable comparison logic for answer forms rather than dataset-specific patches.

Removed group	Direct	Corr.	Δ Avg
Invalid or conflicting outputs	207	38	0.43
Closed choice / verdict mapping	1,016	20	0.33
Numeric / unit normalization	369	22	0.26
Boundary / format stress rules	138	24	0.21
Symbolic math / intervals	175	19	0.21
Terminal / object extraction	540	15	0.20
String / sequence verification	220	11	0.13

Table 3: Group removal under the same prompt verifier. Δ Avg is the macro accuracy drop when direct decisions from the group are routed back to the prompt verifier.

Failure boundaries. We characterize the remaining errors as cases that should remain with the fallback verifier. Code modules route deterministic comparison cases directly and abstain on ambiguous or underspecified judgments, keeping rule activation conservative. In a manual analysis of 80 remaining disagreement cases, 53 involve annotation or ambiguity issues and 27 are remaining verifier errors.

5 Conclusion

We demonstrate that AUTOVERIFIER converts recurring verifier errors into auditable comparison logic. It learns verifier inductive biases from construction residuals and promotes supported patterns into code modules or prompt guidance. Across four benchmarks, it reaches 93.05% macro accuracy, improves the prompt-only verifier by +1.76 points, and cuts fallback calls by 32.13% without changing model parameters.

Limitations

The current scope is reference-based binary verification with a fixed construction pool and frozen inference interface. Future work should extend the same construction loop driven by recurring verifier errors to richer grading rubrics, multiple reference answers, and domain-specific evaluation standards while preserving support, counterexample, overlap, and replay validation constraints. Future work should also make construction easier to reproduce and maintain with open construction agents, versioned rule libraries, clearer record visualization, and continuous replay tests as new domains or answer formats are added.

Ethical Considerations

AUTOVERIFIER judges candidate answers against references and may be used as a reward component in downstream filtering or training pipelines, so incorrect verifier outputs can mislead data selection when references are incomplete or ambiguous. We keep module activation conservative, retain abstention for deterministic rules, and separate benchmark scores from hard case analysis. Construction and benchmark data should be used only under their licenses and permitted use terms; scientific and medical examples are verification cases, not expert advice. If diagnostic cases are released, they should be anonymized and distributed only under the relevant dataset licenses.

References

- Jonathan Baxter. 2000. [A model of inductive bias learning](#). *Journal of Artificial Intelligence Research*, 12:149–198.
- Ding Chen, Qingchen Yu, Pengyuan Wang, Mengting Hu, Wentao Zhang, Zhengren Wang, Bo Tang, Feiyu Xiong, Xinchu Li, Chao Wang, Minchuan Yang, and Zhiyu Li. 2025a. [xVerify: Efficient answer verifier for reasoning model evaluations](#). *Preprint*, arXiv:2504.10481.
- Nuo Chen, Zhiyuan Hu, Qingyun Zou, Jiaying Wu, Qian Wang, Bryan Hooi, and Bingsheng He. 2025b. [JudgeLRM: Large reasoning models as a judge](#). *Preprint*, arXiv:2504.00050.
- DeepSeek-AI. 2025. [DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning](#). *Preprint*, arXiv:2501.12948.
- Yaxin Du, Xiyuan Yang, Zhifan Zhou, Wanxu Liu, Zixing Lei, Zimeng Chen, Fenyi Liu, Haotian Wu, Yuzhu Cai, Zexi Liu, Xinyu Zhu, WenHao Wang, Linfeng Zhang, Chen Qian, and Siheng Chen. 2026. [DataMaster: Data-centric autonomous AI research](#). *Preprint*, arXiv:2605.10906.
- Ruixiang Feng, Zhenwei An, Yuntao Wen, Ran Le, Yiming Jia, Chen Yang, Zongchao Chen, Lisi Chen, Shen Gao, Shuo Shang, Yang Song, and Tao Zhang. 2025. [CoSineVerifier: Tool-augmented answer verification for computation-oriented scientific questions](#). *Preprint*, arXiv:2512.01224.
- Zhezhen Hao, Hong Wang, Jian Luo, Jianqing Zhang, Yuyan Zhou, Qiang Lin, Can Wang, Hande Dong, and Jiawei Chen. 2026. [ReCreate: Reasoning and creating domain agents driven by experience](#). *Preprint*, arXiv:2601.11100.
- Yuzhen Huang, Weihao Zeng, Xingshan Zeng, Qi Zhu, and Junxian He. 2025. [From accuracy to robustness: A study of rule- and model-based verifiers in mathematical reasoning](#). *Preprint*, arXiv:2505.22203.
- Hynek Kydlíček. 2025. [Math-Verify: Math verification library](#). Software library.
- Shudong Liu, Hongwei Liu, Junnan Liu, Linchen Xiao, Songyang Gao, Chengqi Lyu, Yuzhe Gu, Wenwei Zhang, Derek F. Wong, Songyang Zhang, and Kai Chen. 2025a. [CompassVerifier: A unified and robust verifier for LLMs evaluation and outcome reward](#). *Preprint*, arXiv:2508.03686.
- Zijun Liu, Peiyi Wang, Runxin Xu, Shirong Ma, Chong Ruan, Peng Li, Yang Liu, and Yu Wu. 2025b. [Inference-time scaling for generalist reward modeling](#). *Preprint*, arXiv:2504.02495.
- M-A-P Team. 2025. [SuperGPQA: Scaling LLM evaluation across 285 graduate disciplines](#). *Preprint*, arXiv:2502.14739.
- Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. 2026. [Skill-Pro: Learning reusable skills from experience via Non-Parametric PPO for LLM agents](#). *Preprint*, arXiv:2602.01869.
- Alexander Novikov, Ngán Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. [AlphaEvolve: A coding agent for scientific and algorithmic discovery](#). *Preprint*, arXiv:2506.13131.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. [Mathematical discoveries from program search with large language models](#). *Nature*, 625:468–475.

- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. [DeepSeekMath: Pushing the limits of mathematical reasoning in open language models](#). *Preprint*, arXiv:2402.03300.
- TIGER-Lab. 2025. [TIGER-Lab/verifier-sft-data](#). Hugging Face dataset.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. [Voyager: An open-ended embodied agent with large language models](#). *Preprint*, arXiv:2305.16291.
- Jiayi Weng. 2026. [Learning beyond gradients](#). Blog post.
- Yu Xia, Jingru Fan, Weize Chen, Siyu Yan, Xin Cong, Zhong Zhang, Yaxi Lu, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2025. [AgentRM: Enhancing agent generalization with reward modeling](#). *Preprint*, arXiv:2502.18407.
- Yuchen Yan, Jin Jiang, Zhenbang Ren, Yijun Li, Xudong Cai, Yang Liu, Xin Xu, Mengdi Zhang, Jian Shao, Yongliang Shen, Jun Xiao, and Yuet-ing Zhuang. 2025. [VerifyBench: Benchmarking reference-based reward systems for large language models](#). *Preprint*, arXiv:2505.15801.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, and 16 others. 2025. [DAPO: An open-source LLM reinforcement learning system at scale](#). *Preprint*, arXiv:2503.14476.
- Xiang Yue, Tuney Zheng, Ge Zhang, and Wenhui Chen. 2024. [MAmmoTH2: Scaling instructions from the web](#). *Preprint*, arXiv:2405.03548.
- Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. 2025a. [SkillWeaver: Web agents can self-improve by discovering and honing skills](#). *Preprint*, arXiv:2504.07079.
- Shenghe Zheng, Chenyu Huang, Fangchen Yu, Junchi Yao, Jingqi Ye, Tao Chen, Yun Luo, Ning Ding, Lei Bai, Ganqu Cui, and Peng Ye. 2025b. [SCL-Verifier: Scientific verifier with thinking](#). *Preprint*, arXiv:2509.24285.

Appendix Overview

The appendix records implementation, audit, and additional evaluation details that support the main text.

Part	Contents
A	Section A. Illustrative construction cases and rule card anatomy. Anonymized support patterns and compact rule card contracts for terminal extraction, unit normalization, verdict guards, and invalid output guards.
B	Section B. Construction implementation and records. Construction pipeline, model configuration, data pool composition, leakage audit, round trace, rule revision case, support records, and prompt templates.
C	Section C. Additional evaluation, controls, and diagnostics. Statistical uncertainty, direct decision audit, group removal, heuristic control, promotion check ablation, and the Qwen3-4B fallback diagnostic.
D	Section D. Audit, cost, and responsible artifact notes. Manual error analysis, construction and inference cost accounting, artifact access policy, and screening notes.

A Illustrative Construction Cases and Rule Card Anatomy

This section presents case evidence for construction. It starts from paraphrased construction support examples, then shows the rule card contract that records recurring verifier errors and admits code modules through replay validation. The boxed examples are deliberately compact so that the appendix remains in the ACL two column format.

Construction Case 1. Unit Implied Numeric Equivalence

Training source. TIGER construction example.

Fields. The question asks for torque in Newton meters. The reference is 6.79Nm, while the candidate gives the same terminal value as the bare number 6.79.

Why the table form was insufficient. A table entry saying “numeric/unit normalization” hides the actual scale choice. The verifier may accept the missing unit only when the task fixes the unit and the candidate supplies a unique terminal scalar.

Rule action. The accepted rule extracts the terminal scalar, checks that the question supplies the omitted unit, and returns CORRECT only for the supported unit preserving comparison. Unsupported units, multiple final numbers, or unparsed expressions trigger ABSTAIN.

Construction Case 2. Closed Choice Polarity Conflict

Training source. VAR construction example.

Fields. The task asks for a Yes or No judgment. The reference is Yes, while the candidate’s final answer is No after a longer explanation.

Why this belongs in a case box. The error is not visible from aggregate counts. The verifier must align the final answer with the task’s closed vocabulary rather than accept explanatory text around an opposite terminal label.

Rule action. The closed choice rule extracts the unique terminal label from both sides and returns INCORRECT on the polarity conflict. If a response lacks a unique terminal label or mixes equally strong labels, the rule abstains.

Construction Case 3. Short Answer Slot Mismatch

Training source. VAR construction example.

Fields. The task asks for a named creator. The reference is one named person, while the candidate gives a step-by-step lookup response and commits to a different person.

Why this case matters. The candidate contains plausible explanatory text, but the requested output is a compact short answer object. This is a different failure mode from numeric normalization. The verifier must reject process descriptions or wrong objects when the task asks for a compact final object.

Rule action. The short answer guard detects schema, workflow, or process outputs in short answer slots and returns INCORRECT when the required final object is absent or replaced by a different terminal object. It abstains on genuine alias or synonym questions that require semantic knowledge outside the deterministic guard.

Rule Card Contract Encoded by the Code Module

Activation. Fire only after terminal object extraction identifies a supported answer object, such as a numeric speed ratio or a closed vocabulary verdict.

Procedure. Normalize the extracted objects, preserve task-level constraints such as units or verdict vocabularies, and compare only the bounded final answer region.

Abstention condition. Abstain on missing terminal answers, conflicting final values, unsupported units or vocabularies, and expressions that cannot be parsed into the expected object type.

Counterexamples. Reject or abstain on values outside tolerance, incompatible final objects, omitted final answers, and responses that mix a correct intermediate object with a different final answer.

Promotion evidence. Admit the rule only after support examples trigger the intended path, counterexamples abstain or match construction labels, overlap checks add new coverage, and full construction replay introduces no direct decision regression.

B Construction Implementation and Records

This section records the implementation details needed to audit how recurring verifier errors become verifier behavior. It includes the construction pipeline, construction pool composition, per round growth trace, representative rule card contracts, and construction agent prompt protocols.

B.1 Construction Implementation Details

Table 4 summarizes the construction round pipeline and the audit contract for each stage.

The implementation follows the pipeline in Table 4. It initializes an empty module library and a base prompt verifier, then replays the current verifier, clusters recurring verifier errors, drafts rule cards, audits code cards, and promotes only candidates that pass support, counterexample, overlap, and full replay validation. It repairs or merges existing code modules when a cluster exposes an existing module gap, adds a new code module only for uncovered deterministic behavior, incorporates cards that require model judgment into prompt guidance, and freezes the module order, prompt, parser, routing order, and output contract. The design adapts recurring verifier errors and procedural memory to non-parametric verifier optimization with replay validation. The empirical claim remains restricted to the frozen reference-based answer verification experiments reported in the paper.

B.2 Construction Data Details

Construction data is converted to the canonical $(q, a^*, \hat{a}, y)$ schema. VAR train (Chen et al., 2025a), the TIGER-Lab verifier-sft-data dataset (TIGER-Lab, 2025), and SuperGPQA-Records (M-A-P Team, 2025) are natively compatible after field parsing. WebInstruct-verified (Yue et al., 2024) is used only as a source of questions with reference answers; derived verifier examples are constructed by generating candidate responses and assigning binary labels. Construction samples are decontaminated against evaluation sources by exact and fuzzy question/reference matching before code module construction.

The construction procedure follows verifier data construction patterns used by recent verifier work, but it is specialized for deterministic rule induction rather than neural fine tuning. First, all source examples are normalized into a shared verifier schema and assigned source, answer object, and candidate surface metadata. Second, exact and fuzzy decontamination removes examples overlapping the target evaluation questions or references. Third, recurring verifier errors from the fixed construction verifier are grouped by observed error type, such as numeric/unit normalization, symbolic equivalence, terminal extraction, closed choice alignment, string/sequence verification, and invalid output detection. Fourth, candidate rule cards are promoted

only when they have positive support examples, counterexamples, and replay validation with no direct decision regression. We use a single dataset size convention throughout the paper, with the construction pool defined as 5,000 source-derived and constructed examples. Internal validation records are implementation records for rule promotion, not separate data pool definitions.

The construction pool is therefore not a random sample of available records. It is a selected construction pool of 5,000 examples that combines source records with constructed diagnostic examples and balances domain coverage with answer object and error type coverage. For each target error type, the construction pipeline selects positive support examples and counterexamples from the construction pool. The module library is induced only from recurring verifier errors and support examples; every promoted code module must retain live support under promotion validation.

The final frozen construction pool contains source-derived examples and constructed diagnostic examples. The constructed examples exercise malformed outputs, invalid-answer conditions, option or verdict conflicts, ordered answer-list surfaces, terminal answer formats, and other answer-format cases that should be covered before deterministic rules are promoted. Following the leakage-audit practice used in data-centric autonomous research systems, we run a layered construction-evaluation overlap check over exact hashes, fuzzy token overlap, provenance, and 3–5-gram overlap diagnostics (Du et al., 2026). As shown in Figure 4, the final construction pool has no exact question, exact question/reference, exact verifier-triple, or fuzzy verifier-triple matches against the 8,295 evaluation examples; the only nonzero plotted signal is low verifier-triple 5-gram surface overlap. Table 7 reports the corresponding counts and diagnostic-only overlaps.

The runtime support records provide a stricter rule view. The final support set contains 2,096 live source support examples with 0 direct support errors, plus 13 constructed boundary/format validation records, giving 2,109 supported construction examples in the construction monitor over 5,000 examples. The final support records validate all 63 delivered code module behaviors with 0 missing support entries and 0 runtime errors. These support records are construction records for rule promotion, not additional data size conventions.

Step	Operation	Audit contract
1	Normalize construction examples	Convert all sources into $(q, a^*, \hat{a}, y)$ with source, domain, answer object, and surface form metadata.
2	Decontaminate	Remove exact and fuzzy question/reference overlaps with all evaluation benchmarks before rule construction.
3	Collect residuals	Run the base verifier in the first round and the current verifier in later rounds; retain label mismatches in the residual buffer.
4	Cluster errors	Group residuals by answer object and observed error type, rather than by benchmark identity or evaluation labels.
5	Draft rule cards	For each cluster, specify activation, deterministic procedure, termination rule, support examples, and counterexamples.
6	Audit code cards	Compare each code card with the current module library; repair or merge an existing code module when it has the right behavior type but incomplete guards, add a new code module only when no existing path covers the behavior, and send cards that require model judgment to prompt guidance.
7	Promote or reject	Accept a code update only if support, counterexamples, overlap checks, and full construction replay satisfy the direct decision promotion contract.
8	Freeze runtime state	Lock module order, prompts, parsers, routing, and output contracts after the construction rounds.

Table 4: Construction round pipeline. Each accepted update is generated from construction replay on the construction pool and promoted by replay validation before the runtime is frozen.

Component	Model	Reasoning/decoding	Role and evaluation boundary
Rule mining agent	GPT-5.5	xhigh reasoning	Construction only; records recurring verifier error clusters in rule cards.
Code generation agent	GPT-5.5	xhigh reasoning	Construction only; audits code cards against the current library, then proposes repairs, merges, new code modules, and tests.
Prompt optimization agent	GPT-5.5	xhigh reasoning	Construction only; incorporates rule card evidence that requires model judgment into prompt guidance for the sectioned binary prompt.
Fallback verifier V_{fb}	GPT-5.4-Mini	deterministic decoding when supported	Inference/evaluation only on examples for which all code modules abstain; outputs are parsed by a strict binary parser.

Table 5: Model configuration for construction and inference. Construction agents operate only on construction replay records and cannot promote code modules without replay validation; the module library, prompt, parser, and routing order are fixed before reported inference runs.

B.3 Round Trace Records

Figure 3 is generated from the accepted round records. The construction curve uses the single frozen construction pool of 5,000 source-derived and constructed examples, with 2,096 live support examples from source replay plus 13 constructed boundary/format validation examples, for 2,109 covered examples in the construction monitor over 5,000 examples. After the runtime is frozen, we replay the same final module library on the four benchmarks to report direct routing counts of 533 on VerifyBench, 372 on VerifyBench-Hard, 897 on SCI-VerifyBench, and 863 on VerifierBench. The benchmark counts are post-freeze routing summaries, not construction feedback.

This table is an audit record rather than a performance curve. A later round is considered more general when it increases direct coverage on the construction pool through a typed code module behavior that also retains support examples, coun-

terexamples, and a clean construction replay. Candidates that looked useful but lacked construction support or failed counterexample replay remain in the rejected proposal or support gap records; they are not counted as accepted updates.

B.4 Construction Side Rule Revision Case

Table 9 expands one Round 8 identifier revision from the construction records. The original support example is a SuperGPQA corn genotype verifier pair. The reference answer is AaCCRRr, the candidate answer is AaCCrr, and the construction label is INCORRECT. The revision fixes an unsafe compact surface match by preserving case for mixed case scientific identifiers, then promotes a fail rule that fires only when a whole terminal identifier can be extracted from both sides.

Axis	Coverage	Role
Domain/source	VAR 1630; TIGER 1196; SuperGPQA derived 1014; WebInstruct derived 953; constructed diagnostic examples 207	Supplies construction questions and references across math, science, engineering, medicine, general reasoning, instruction following, short answer verification, closed choice verification, and controlled comparison cases.
Answer object	symbolic 1481; numeric 1373; closed choice 1184; short answer 858; open text 77; scientific identifier 13; list/sequence/word set/equation 14	Covers the object types and comparison surfaces that deterministic verifiers must parse before comparing references and candidates.
Error type	closed choice/verdict alignment; numeric/unit normalization; symbolic/interval comparison; terminal extraction; string/sequence verification; invalid or conflicting outputs	Guides error clustering, support examples, and counterexamples for candidate rule cards.

Table 6: Two dimensional construction pool design. The domain and source axis supplies broad task coverage, while the answer object and error type axes ensure that construction examples expose the verifier errors that code modules are meant to solve. Counts are from the selected construction pool of 5,000 source-derived and constructed examples; promotion validation is summarized in the text.

Audit item	Construction side evidence
Construction example	SuperGPQA support example with reference AaCCrr, candidate AaCCrr, and label INCORRECT; internal row identifiers are omitted from the paper.
Observed pattern	Mixed case identifier strings may differ only by case or terminal slots; lowercase compact normalization is unsafe for genotype, alloy, spectral class, and formula like identifiers.
Generalized edit	Add whole terminal mixed case opaque identifier extraction and route different extracted identifiers to a direct fail. The guard abstains if the reference identifier appears anywhere in the candidate.
Same group support	Five SuperGPQA examples support the same group, including AaCCrr/AaCCrr, Na_2HXO_3/Na_3XO_3, M2IIa/M2IIb, ZCuPb30/ZPbSn10Cu5, and ACBacb/AaBcBC.
Replay result	Runtime replay returns INCORRECT with reason code terminal_short_slot_mismatch; clean construction replay has 1,980 direct correct decisions, 0 direct wrong decisions, and 0 runtime errors over 4,997 records.

```

def _is_opaque_identifier_token(token: str) -> bool:
    return bool(
        re.fullmatch(r"[A-Za-z][A-Za-z0-9-]{5,}", token)
        and re.search(r"[A-Z]{2,}", token)
        and re.search(r"[a-z]", token)
    )

reference_token = _terminal_reference_opaque_identifier(reference)
if reference_token is None:
    return None
if reference_token in str(candidate or ""):
    return None
candidate_obj = _terminal_candidate_opaque_identifier(candidate)
if candidate_obj is None:
    return None
candidate_token = str(candidate_obj.get("token") or "")
if candidate_token == reference_token:
    return None

if opaque_identifier_mismatch_evidence is not None:
    return _build_fail_decision(
        reason_code="opaque_identifier_terminal_mismatch",
        evidence=evidence,
    )

```

Table 9. Code module construction audit case for a mixed case identifier revision. The selected source lines are copied from the frozen runtime snapshot and show the activation guard plus the direct fail routing path.

B.5 Module Library Support

The final module library contains 63 supported code module behaviors with selected pool or fix-

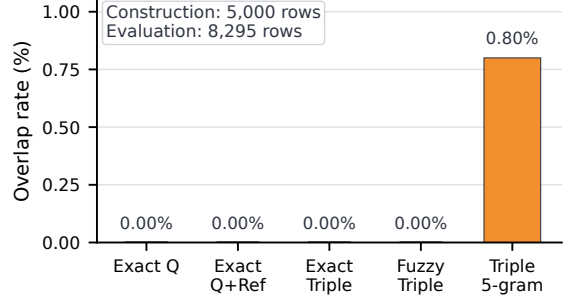


Figure 4: Construction–evaluation leakage check for the frozen construction pool. In the contamination-audit format used by data-centric autonomous research systems, the decisive exact and fuzzy verifier-triple checks are zero; the only nonzero plotted signal is low verifier-triple 5-gram surface overlap. Diagnostic question–reference and reference-only repeats are reported in Table 7 because short answers and templates can repeat without copying full verifier examples.

ture based support. Support is verified by replaying the accepted code modules on support tests outside evaluation. No direct errors are observed, and all 63 target behaviors have at least one live runtime support hit in the final closure pool. The largest supported groups include compact exact object extraction, closed vocabulary verdict matching, numeric and unit normalization, single choice terminal mismatch detection, interval endpoint mismatch detection, and invalid answer guards.

Each rule card records an activation condition, deterministic execution logic, abstention conditions, support examples, and counterexamples, following the boxed contract in Section A. Code modules are promoted only when support examples trigger the intended reason, counterexamples abstain or return the construction label as expected, and full construction replay shows no direct error regression. Rejected rule cards are retained as construction feedback when they are too broad, overlap an existing module without coverage gain, or rely on semantic assumptions that cannot be checked deterministically.

B.6 Prompt Templates

The rule mining prompt consumes recurring verifier errors, answer object metadata, support hints, and counterexamples, and must emit structured rule card candidates with activation, procedure, termination, support examples, counterexamples, and rejection risks. The code generation prompt consumes an accepted rule card plus parser utilities and replay constraints, and must emit the narrowest code module that returns CORRECT, INCORRECT,

Audit check	Field / threshold	Count	Interpretation
Exact question hash	Normalized question	0	No construction example copies an evaluation question.
Exact question–reference hash	Normalized question + reference	0	No exact evaluation question–answer pair appears in construction.
Exact verifier-triple hash	Question + reference + candidate	0	No exact evaluation verifier example appears in construction.
Fuzzy verifier-triple match	Token Jaccard ≥ 0.90	0	No near-duplicate verifier example is detected under the strict triple check.
Fuzzy question–reference match	Token Jaccard ≥ 0.90	84	Template-level overlap in task wording; retained as diagnostic because the stricter verifier-triple match is zero.
Reference-only exact match	Normalized reference	64,091	Diagnostic only: short answers repeat across verifier tasks and are not treated as leakage without question or candidate overlap.
Verifier-triple 5-gram overlap	Unique 5-grams	0.80%	Low surface overlap diagnostic over full verifier triples.

Table 7: Construction–evaluation leakage audit for the final construction pool of 5,000 examples against 8,295 benchmark evaluation examples. Exact checks use normalized hashes; fuzzy checks use token-Jaccard matching at threshold 0.90.

or ABSTAIN. The prompt optimization prompt consumes accepted and rejected rule contracts plus prompt verifier errors, then extracts transferable comparison principles into the same sectioned binary prompt used after module abstention.

The AUTOVERIFIER prompt verifier uses Role, Rules, Steps, and Output Contract blocks; it is not an insertion into the official benchmark prompts, which are used only as baselines. Benchmark rendering receives the question, reference answer, and candidate answer, and it is always evaluated as a binary classifier over CORRECT and INCORRECT; there is no third prompt label. Raw construction examples are not inserted into the evaluation prompt. Output parsing maps explicit correctness labels to the binary interface and treats unparseable outputs as incorrect under reported evaluation.

C Additional Evaluation, Controls, and Diagnostics

This section keeps only the evaluation views needed for reviewer interpretation. These views are uncertainty estimates, direct decision audit, controls, promotion check ablations, and fallback diagnostics. Code module coverage is summarized in the main text; longer category breakdowns, direct decision confusion matrices, and fallback error slices are omitted because they restate the same routed examples at finer granularity.

Representative construction support cases are shown in gray boxes in Section A. The omitted per benchmark category slices, fallback error concentration, and direct decision group tables are diagnostic views of the same routed examples rather than separate evidence needed to understand the main claim.

C.1 Statistical Uncertainty

Table 10 reports uncertainty for the AUTOVERIFIER prompt-only and prompt-plus-code variants under the same final prompt. Accuracy intervals use Wilson intervals, and Δ intervals use a paired normal approximation over per example correctness differences. Under this paired comparison, all four code module gains over the prompt component are positive.

We do not repeat overall F1, balanced accuracy, MCC, direct decision confusion matrices, and group level Wilson intervals because they restate the same paired evaluation records at a finer granularity; the uncertainty summary in Table 10 and the direct audit in Table 11 are the necessary checks for the appendix.

Table 11 reports the direct decision audit behind the final evaluation records. It lists direct coverage, prompt only errors corrected by code modules, direct decision label distribution, and Wilson lower bounds for observed direct accuracy. The per example records contain module predicted CORRECT/INCORRECT counts, benchmark label counts, and per label precision/recall.

C.2 Additional Controls and Diagnostics

The additional checks target five alternatives a reviewer may consider. The gain might come from prompt rewriting rather than code modules, be reproducible with generic rule heuristics, rely on one dominant module group, depend on weak replay validation, or be tied to the proprietary GPT fallback. The same prompt comparison addresses the first alternative. The AUTOVERIFIER component that uses only the prompt improves the four benchmark average over official GPT-5.4-Mini prompts from 91.03 to 91.29 (+0.26), although individ-

Round	Generalized construction scope	Contracts / groups	Construction coverage	Audit interpretation
0	No code modules	0 / 0	0 $\rightarrow$ 0	Baseline with all construction examples routed to the fallback verifier.
1	Exact object and terminal slot rules	5 / 5	+1,117 $\rightarrow$ 1,117	Repeated exact terminal objects are consolidated into reusable activation regions instead of exceptions tied to individual examples.
2	Closed option terminal rules	12 / 12	+144 $\rightarrow$ 1,261	Option rules generalize over recoverable option maps and terminal option surfaces, with nearby option conflicts retained as counterexamples.
3	Closed vocabulary verdict and polarity rules	5 / 5	+102 $\rightarrow$ 1,363	Verdict and polarity phrases are promoted as typed closed vocabulary comparators rather than free form string matches.
4	Numeric, unit, and scientific notation rules	16 / 16	+315 $\rightarrow$ 1,678	Numeric rules add tolerance, unit, and notation boundaries only when construction replay preserves label aligned direct decisions.
5	Symbolic, formula, and interval rules	13 / 13	+162 $\rightarrow$ 1,840	Algebraic and interval rules broaden comparison beyond exact text while keeping endpoint, set, and surface form counterexamples.
6	Sequence, string, and invalid output rules	11 / 11	+70 $\rightarrow$ 1,910	Structured string and invalid answer guards add direct failures for recurring terminal object errors and abstain outside their typed region.
7	Terminal no answer plus angle/ratio support	8 / 8	+40 $\rightarrow$ 1,950	The audit record keeps narrow terminal non answer, angle, and ratio rules after support reselection and construction replay, not benchmark only hits.
8	Unit rate and opaque identifier rules	4 / 4	+45 $\rightarrow$ 1,995	Identifier and SI rate rules are admitted only with whole terminal extraction and unit context guards, turning earlier boundary fixes into reusable rules.
9	Terminal phrase and short phrase expansions	4 / 3	+54 $\rightarrow$ 2,049	Phrase rules add bounded terminal window containment; broader substring candidates remain rejected or routed to fallback.
10	Scientific relative, prefix, unit bare, and boundary/format rules	5 / 5	+60 $\rightarrow$ 2,109	The final checkpoint closes the construction monitor used by the figure; the separate support closure replay validates all 63 delivered code module behaviors.

Table 8: Construction generalization audit trace for the ten accepted optimization rounds. Coverage is measured only on the frozen construction monitor used by Figure 3; benchmark examples are not used to promote a code module. Each row records the generalized rule scope that entered the module library after support, counterexample, overlap, and construction replay validation.

Benchmark	Rows	Prompt	AUTOVERIFIER	Δ	95% CI for Δ
VerifyBench	2000	96.20 [95.27, 96.95]	96.75 [95.88, 97.44]	+0.55	[+0.23, +0.87]
VerifyBench-Hard	978	90.49 [88.49, 92.17]	92.43 [90.60, 93.93]	+1.94	[+1.08, +2.81]
SCI-VerifyBench	2500	86.80 [85.42, 88.07]	89.92 [88.68, 91.04]	+3.12	[+2.44, +3.80]
VerifierBench	2817	91.66 [90.58, 92.62]	93.11 [92.12, 93.99]	+1.45	[+1.01, +1.90]

Table 10: Statistical uncertainty for AUTOVERIFIER prompt and prompt plus code results under the same prompt verifier. Confidence intervals for Δ use a paired normal approximation over per example correctness differences.

ual benchmark deltas are mixed. The deltas are -0.10 on VerifyBench, +0.20 on VerifyBench-Hard, +0.04 on SCI-VerifyBench, and +0.89 on VerifierBench. Adding code modules to the same prompt verifier then improves VerifyBench from 96.20 to 96.75 (+0.55), VerifyBench-Hard from 90.49 to 92.43 (+1.94), SCI-VerifyBench from 86.80 to 89.92 (+3.12), and VerifierBench from 91.66 to 93.11 (+1.45). The four benchmark average improves from 91.29 to 93.05 (+1.76) after adding code modules. This paired comparison fixes both the fallback model and the final prompt, so the gain is attributed to code modules rather than to another prompt change.

Table 3 in the main text reports an offline removal ablation under the same prompt verifier. For each group, direct decisions from that group are replaced by the prompt verdict, while all other code

module decisions and prompt calls are unchanged. This isolates the marginal contribution of each module group without introducing additional model calls. A group can therefore have high direct coverage but a modest accuracy drop if most of its direct decisions were already correct under the prompt verifier; the corrected count measures the stricter contribution to accuracy. High value code modules are those that remove systematic prompt failures, not necessarily those that fire most often.

The per benchmark removal and cumulative routing views are omitted from the appendix table set because they are deterministic decompositions of the same evaluation records. Invalid or conflicting output guards matter most on SCI-VerifyBench, closed choice and verdict mapping matter most on VerifyBench-Hard and VerifierBench, and the cumulative route reaches 2,665 direct decisions,

Benchmark	Direct	P-corr.	P-err. fixed	M-err.	Eq.	Not-eq.	Wilson LB
VerifyBench	533	522	11	0	353	180	99.28
VerifyBench-Hard	372	353	19	0	131	241	98.98
SCI-VerifyBench	897	819	78	0	571	326	99.57
VerifierBench	863	822	41	0	621	242	99.56
All	2,665	2,516	149	0	1,676	989	99.86

Table 11: Direct decision audit under the fixed final verifier. P-err. fixed counts direct decisions for which the AUTOVERIFIER prompt verifier is wrong and the code module is correct. M-err. counts code module disagreements with the benchmark label. Wilson LB is the lower bound of a two sided 95% Wilson interval for direct accuracy.

32.13% fallback call reduction, and 149 prompt errors corrected after all promoted code modules are enabled.

Table 12 adds a small frozen rule control. It uses only generic exact final answer matching, single choice option comparison, closed vocabulary verdict comparison, and exact numeric equality checks, then falls back to the same prompt verifier on all other examples. The control covers fewer examples than code modules learned from recurring verifier errors, has nonzero direct errors, and slightly reduces final micro accuracy relative to the prompt verifier. This negative control is deliberately conservative. It tests whether the improvement can be explained by obvious deterministic shortcuts before crediting the construction process. Its regressions show why AUTOVERIFIER promotes code modules with abstention guards admitted by replay validation instead of always applying broad exact matching heuristics.

Table 13 reports an archived candidate promotion check ablation. We collect 200 generated candidate code modules from saved construction records, including support checks, counterexample checks, construction replay summaries, protected construction-slice summaries, and candidate check reports. The strict policy reproduces the full promotion contract on these archived candidates. We then weaken one acceptance condition at a time and measure the direct errors or construction violations that would have entered the module library. This experiment audits the promotion checks over generated candidates using saved construction records, rather than reconstructing alternate checks from scratch. The important pattern is that weaker checks accept more candidates but also admit errors. Removing full replay accepts 51 candidates and admits 17 replay errors, while promotion using support alone accepts 95 candidates but admits replay, protected-slice, and counterexample failures. Thus the promotion check ablation supports precision preserving promotion rather than merely

showing that the strict policy is conservative.

Table 14 reports a public 4B class fallback diagnostic over the full benchmark suite. We keep the module library, routing records, parser, and direct decisions fixed, replace only the fallback verifier with local Qwen3-4B, and evaluate all 8,295 benchmark examples. The run uses one record per context, strict binary parsing, and counts unparseable fallback outputs as incorrect. This diagnostic checks whether the same code module protocol improves a fallback model outside GPT under fixed examples and parser settings.

Group removal, the frozen rule control, the archived candidate promotion check ablation, and the full Qwen3-4B fallback diagnostic are the completed checks used for the current accuracy attribution and robustness diagnosis. The promotion check ablation supports the practical need for protected construction-slice replay, precision checks, counterexamples, and full replay over archived generated candidates. The Qwen3-4B diagnostic shows that the same code modules improve a public 4B fallback from 89.31 to 91.62 macro accuracy while avoiding 32.13% of fallback calls. We treat this as a diagnostic of replacing the fallback, not as a claim that the rule library is invariant across all fallback models.

The official prompt baseline uses the public benchmark prompt format with GPT-5.4-Mini and the same binary label parser. Under the same binary evaluation protocol, the official prompt runs reach 96.30% on VerifyBench full, 90.29% on VerifyBench-Hard, 86.76% on SCI-VerifyBench, and 90.77% on VerifierBench.

D Audit and Cost

This section keeps the audit material needed to interpret the reported numbers. It includes the manual error analysis and construction versus inference cost accounting.

Benchmark	Rows	Direct	Direct Corr.	Coverage	Prompt Acc.	Heur.+Prompt Acc.	Reg.
VerifyBench	2,000	278	96.76	13.90	96.20	95.80	9
VerifyBench-Hard	978	224	95.98	22.90	90.49	90.18	6
SCI-VerifyBench	2,500	0	—	0.00	86.80	86.80	0
VerifierBench	2,817	0	—	0.00	91.66	91.66	0
All	8,295	502	96.41	6.05	91.15	91.02	15

Table 12: Frozen rule heuristic control. The control uses only generic exact normalized final answer match, single choice option comparison, closed vocabulary verdict comparison, and exact numeric equality, with the same prompt verifier as fallback. Reg. counts examples where the prompt verifier was correct but the rule direct decision is wrong. This control performs no model calls and does not use generated AUTOVERIFIER code modules.

Promotion policy	Eval.	Accept	Replay	Replay err.	Slice err.	Slice fail	Ctr-ex
Full checks	200	20	20	0	0	0	0
w/o protected replay	102	28	28	0	9	8	0
w/o precision regression check	102	23	23	3	0	0	0
w/o counterexample check	88	14	14	0	0	0	21
w/o full replay	186	51	32	17	9	3	0
Support only	186	95	55	36	22	5	286

Table 13: Archived candidate promotion check ablation over saved construction records. Eval. counts candidates evaluable under the weakened policy; Replay counts accepted candidates with saved full replay summaries. Error columns count replay, protected-slice, and counterexample violations admitted by each policy.

D.1 Manual Error Analysis

We conduct the manual error analysis after removing all direct code module decisions. For each sampled disagreement, the authors inspect the question, reference answer, candidate answer, benchmark label, and model verdict, without seeing the rule reason or construction history. Each case is assigned to one of four diagnostic categories: true verifier error, ambiguous or underspecified reference, multiple valid answers, or likely label/reference issue. An additional author reviews each assignment and records a natural language justification and, for math/science cases, a normalized object comparison when possible. The analysis covers 80 remaining disagreement cases, 20 from each benchmark, and stores one final diagnostic category per case, summarized in Figure 5. We use this analysis for hard case diagnosis; benchmark scores continue to use the original benchmark labels, and diagnostic counts are excluded from rule promotion.

The diagnostic categories are true verifier error, ambiguous or underspecified reference, multiple valid answers, and likely label/reference issue. The accompanying analysis report also records the longer taxonomy of reference sensitive disagreements, cases involving implicit assumptions in annotations, sparse reasoning failures, domain external equivalence, and mixed or overlong responses. These categories are diagnostic only. Benchmark scores continue to use the original benchmark labels, and diagnostic labels are never fed back into rule promotion.

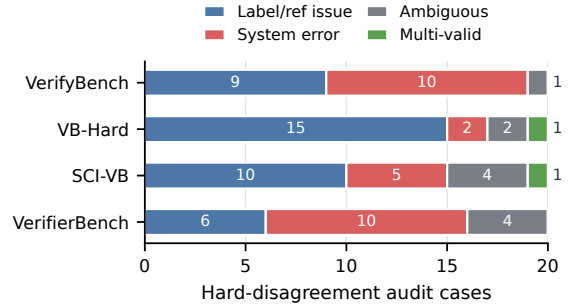


Figure 5: Manual error analysis of 80 remaining disagreement cases. Each benchmark contributes 20 cases. Counts are diagnostic only and are not used for construction round rule updates or prompt revision.

D.2 Cost and Audit Accounting

Table 15 separates construction time accounting from inference time savings. Code modules replace fallback calls on direct decisions after the verifier is frozen; construction agent calls are counted separately from reported inference calls.

D.3 Responsible Artifact and Checklist Notes

Table 16 summarizes the artifact access policy used for this work. We use public or derived sources only for verifier construction, benchmark evaluation, and audit records. The supplementary package contains an artifact README, requirements file, prompt templates, anonymized synthetic examples, and alignment notes; it should contain derived schemas, hashes, code modules, prompts, routing ledgers, and aggregate audit records rather than redistributing raw third-party examples unless their original licenses and terms permit redistribution.

Benchmark	Rows	Qwen3-4B prompt	Direct decisions	Calls↓	Prompt+Code
VerifyBench	2,000	95.40	533	26.65	96.40
VerifyBench-Hard	978	90.39	372	38.04	92.43
SCI-VerifyBench	2,500	83.64	897	35.88	86.44
VerifierBench	2,817	87.82	863	30.64	91.20
Macro avg.	—	89.31	—	—	91.62
All examples	8,295	88.69	2,665	32.13	91.16

Table 14: Full Qwen3-4B open fallback diagnostic. The module library, routing records, parser, and direct decisions are fixed; only the fallback verifier is swapped from GPT-5.4-Mini to local Qwen3-4B. Prompt+Code routes direct decisions through the same code modules and sends abstained examples to Qwen3-4B. Calls↓ is the percentage of examples that avoid a Qwen fallback call. Macro averages are unweighted over the four benchmarks; All examples are size weighted.

Component	Model/code	Rows/calls	Calls saved	Accounting note
Construction agents	GPT-5.5 xhigh	Offline calls	—	Construction only rule mining, code generation, and prompt optimization. These calls are counted separately from inference and are not included in fallback-call reduction.
Submission session cost	Codex submission sessions	6.97B tokens	—	The submission sessions record 6.71B cached input tokens and 18.77M output tokens. Under standard API token prices, this corresponds to approximately \$765.65 if charged as GPT-5.4-Mini, \$2.55K as GPT-5.4, or \$5.10K as GPT-5.5.
Prompt only inference	GPT-5.4-Mini	8,295 fallback calls	0	Controlled baseline with every benchmark example sent to the binary prompt verifier.
Prompt + Code inference	GPT-5.4-Mini + code modules	5,630 fallback calls	2,665	Final verifier: code modules route 2,665 examples directly, reducing fallback calls by 32.13%.
Open fallback diagnostic	Qwen3-4B + code modules	5,630 fallback calls	2,665	Same code module records and parser; only the fallback verifier is swapped.
Direct code execution	Runtime code	2,665 direct decisions	—	Direct decisions have no model token cost at inference. Runtime code, route, rule id, prompt verdict, and final verdict are recorded in the evaluation records.
Evaluation audit	Runtime records	All reported examples	—	Recomputes macro/micro accuracy, direct decision confusion, fallback calls, and paired prompt vs code deltas from the frozen evaluation records.

Table 15: Cost and audit accounting. Construction calls are counted separately from inference calls; inference efficiency is reported as fallback call reduction after the verifier is frozen.

Artifact class	Examples	Use in this paper	Access and release policy
Construction sources	VAR train, TIGER-Lab verifier-sft-data, SuperGPQA-Records, WebInstruct-verified questions	Construction replay, support examples, and counterexamples	Governed by the original source licenses and access terms. We do not assert a new license over raw source records; derived manifests and hashes can be shared when redistribution of raw examples is not permitted.
Constructed diagnostics	Boundary, format, invalid-output, option-conflict, and terminal-answer cases	Construction support and replay validation	Generated verifier examples are released only after removing identifying strings and only with enough metadata to reproduce support and counterexample roles.
Evaluation benchmarks	VerifyBench, VerifyBench-Hard, SCI-VerifyBench, and VerifierBench	Frozen benchmark evaluation	Benchmark data remains under the original benchmark licenses and terms. Evaluation ledgers report benchmark identifiers, routes, module ids, prompt verdicts, final verdicts, parser status, and hashes needed to recompute reported tables.
Verifier records	Code modules, prompts, parser settings, priority order, and route ledgers	Reconstructing reported routing, direct decisions, and fallback calls	The release package should exclude API keys, absolute local paths, author-identifying metadata, and non-anonymous repository links. Model calls to proprietary services are documented by model name and role rather than by undisclosed model parameters.
Manual audit records	Author audit of 80 remaining disagreement cases	Diagnostic taxonomy only	We report aggregate categories in the paper. Case-level diagnostic records should be released only when source licenses allow and after redacting identifying or sensitive text fields.

Table 16: Artifact access, intended use, and release policy. The table distinguishes raw third-party sources from derived records that are sufficient for auditing the reported verifier behavior.

Before sharing derived records, we screen released text fields for personally identifying strings, contact information, author or local path metadata, and offensive content that is not necessary for verifier auditing. Flagged fields are removed, redacted, or replaced by hashes when the raw text is not

needed to recompute a reported result. The construction and evaluation records are intended for reference-based verifier research and audit replication, not for high-stakes grading, medical advice, or deployment as an expert system.

The manual error analysis in Section D.1 is an

internal author audit rather than a recruited human-subjects annotation study. No external annotators are recruited, no demographic attributes are collected, and the audit labels are not used for rule promotion or benchmark relabeling. If future work uses external annotators, it should report task instructions, recruitment, compensation, consent, and institutional review or exemption status in the artifact documentation.